



Final Project Presentation

**how age effects the
performance of different
baseball positions at the
professional level?**

—

Importing from SQL

```
5]: cursor.execute("show tables")
for x in cursor:
    print(x)
```

```
('appearances',)
('city',)
('country_language',)
('fielding',)
('fielding_outfield',)
('fielding_postseason',)
('flight_data',)
('house_data_clf',)
('house_data_reg',)
('Movie',)
('player',)
('salary',)
```

```
6]: player_data
```

```
6]:
```

	player_id	birth_year	name_first	name_last	name_given	weight	height	bats	throws	retro_id	bbref_id
0	0	1981			David	0	0	2	7	2004-04-06	2015-08-23
1	aardsda01	1981	David	Aardsma	David Allan	220	75	R	R	aardd001	aardsda01
2	aaronha01	1934	Hank	Aaron	Henry Louis	180	72	R	R	aaroh101	aaronha01
3	aaronto01	1939	Tommie	Aaron	Tommie Lee	190	75	R	R	aarot101	aaronto01
4	aasedo01	1954	Don	Aase	Donald William	190	75	R	R	aased001	aasedo01
...	...	...	...	...	...	...	...	...	...	...	...
17721	zupofr01	1939	Frank	Zupo	Frank Joseph	182	71	L	R	zupof101	zupofr01
17722	zuvelpa01	1958	Paul	Zuvela	Paul	173	72	R	R	zuvep001	zuvelpa01
17723	zuverge01	1924	George	Zuverink	George	195	76	R	R	zuveg101	zuverge01
17724	zwilldu01	1888	Dutch	Zwilling	Edward Harrison	160	66	L	L	zwild101	zwilldu01
17725	zychto01	1990	Tony	Zych	Anthony Aaron	190	75	R	R	zycht001	zychto01

17726 rows x 11 columns

```
98]: fielding_data
```

```
98]:
```

	player_id	year	stint	team_id	league_id	pos	g	gs	inn_outs	put
0	aardsda01	2004	1	SFN	NL	P	11		33.0	
1	aardsda01	2006	1	CHN	NL	P	45		159.0	
2	aardsda01	2007	1	CHA	AL	P	25		96.0	
3	aardsda01	2008	1	BOS	AL	P	47		147.0	
4	aardsda01	2009	1	SEA	AL	P	73		213.0	
...	...	...	...	...	...	...	...	...	...	...
38736	zuvelpa01	1987	1	NYA	AL	2B	7	6.0	168.0	
38737	zuvelpa01	1988	1	CLE	AL	SS	49	44.0	1139.0	
38738	zuvelpa01	1989	1	CLE	AL	3B	5	5.0	131.0	
38739	zuvelpa01	1991	1	KCA	AL	3B	2	0.0	6.0	
38740	zychto01	2015	1	SEA	AL	P	13			

38741 rows x 18 columns

Joining Tables

```
j: #remove unnecessary player data from table
   columns_to_drop = ['weight', 'height', 'bats', 'throws', 'retro_id', 'bbref_id']
   joined_data.drop(columns=columns_to_drop, inplace=True)
```

```
j: joined_data
```

```
j:
      player_id  year  stint  team_id  league_id  pos   g   gs  inn_outs  putouts  assists  errors  double_plays  passed_balls  wild_pitches  opponent_stolen_bas
0  aardsda01  2004    1    SFN      NL      P   11      33.0    0    0    0    0
1  aardsda01  2006    1    CHN      NL      P   45    159.0    1    5    0    1
2  aardsda01  2007    1    CHA      AL      P   25    96.0    2    4    1    0
3  aardsda01  2008    1    BOS      AL      P   47   147.0    3    6    0    0
4  aardsda01  2009    1    SEA      AL      P   73   213.0    2    5    0    1
...      ...    ...    ...    ...      ...    ...    ...    ...    ...    ...    ...    ...
38736  zuvelpa01  1987    1    NYA      AL    2B    7    6.0   168.0   17   18    0    5
38737  zuvelpa01  1988    1    CLE      AL    SS   49   44.0  1139.0   77  112    8   21
38738  zuvelpa01  1989    1    CLE      AL    3B    5    5.0   131.0    4    8    1    0
38739  zuvelpa01  1991    1    KCA      AL    3B    2    0.0    6.0    0    0    0    0
38740  zychto01  2015    1    SEA      AL    P   13      0    3    0    0
```

38741 rows x 20 columns

Making it 'Modern'

```
joined_data = joined_data[joined_data['year'] >= 1985]
```

```
19]: joined_data
```

```
19]:
```

	player_id	year	stint	team_id	league_id	pos	g	gs	inn_outs	putouts	...	errors	double_plays	passed_balls	wild_pitches	opponent_stolen_bases	ca
--	-----------	------	-------	---------	-----------	-----	---	----	----------	---------	-----	--------	--------------	--------------	--------------	-----------------------	----

0	aardsda01	2004	1	SFN	NL	P	11		33.0	0	...	0	0				
---	-----------	------	---	-----	----	---	----	--	------	---	-----	---	---	--	--	--	--

1	aardsda01	2006	1	CHN	NL	P	45		159.0	1	...	0	1				
---	-----------	------	---	-----	----	---	----	--	-------	---	-----	---	---	--	--	--	--

2	aardsda01	2007	1	CHA	AL	P	25		96.0	2	...	1	0				
---	-----------	------	---	-----	----	---	----	--	------	---	-----	---	---	--	--	--	--

3	aardsda01	2008	1	BOS	AL	P	47		147.0	3	...	0	0				
---	-----------	------	---	-----	----	---	----	--	-------	---	-----	---	---	--	--	--	--

4	aardsda01	2009	1	SEA	AL	P	73		213.0	2	...	0	1				
---	-----------	------	---	-----	----	---	----	--	-------	---	-----	---	---	--	--	--	--

...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

38736	zuvelpa01	1987	1	NYA	AL	2B	7	6.0	168.0	17	...	0	5				
-------	-----------	------	---	-----	----	----	---	-----	-------	----	-----	---	---	--	--	--	--

38737	zuvelpa01	1988	1	CLE	AL	SS	49	44.0	1139.0	77	...	8	21				
-------	-----------	------	---	-----	----	----	----	------	--------	----	-----	---	----	--	--	--	--

38738	zuvelpa01	1989	1	CLE	AL	3B	5	5.0	131.0	4	...	1	0				
-------	-----------	------	---	-----	----	----	---	-----	-------	---	-----	---	---	--	--	--	--

38739	zuvelpa01	1991	1	KCA	AL	3B	2	0.0	6.0	0	...	0	0				
-------	-----------	------	---	-----	----	----	---	-----	-----	---	-----	---	---	--	--	--	--

38740	zychto01	2015	1	SEA	AL	P	13			0	...	0	0				
-------	----------	------	---	-----	----	---	----	--	--	---	-----	---	---	--	--	--	--

38741 rows x 21 columns

Catchers

```
1: #seperate catchers from the rest of the data
# Filter out instances with position 'C'
c_data = fielding_data_sorted[fielding_data_sorted['pos'] == 'C']

# Remove 'C' instances from fielding_data_sorted
fielding_data_sorted = fielding_data_sorted[fielding_data_sorted['pos'] != 'C']

# Print first few rows of c_data
print("C Data:")
print(c_data.head())

# Print first few rows of fielding_data_sorted after removing 'C'
print("\nRemaining Fielding Data:")
print(fielding_data_sorted.head())
```

C Data:

	pos	g	gs	inn_outs	putouts	assists	errors	double_plays	\
19370	C	9			55	3	1		1
20810	C	64		1290.0	267	32	4		3
20811	C	2		6.0	2	0	0		0
20815	C	3		21.0	4	0	1		0
20816	C	1		3.0	0	0	0		0

	passed_balls	wild_pitches	opponent_stolen_bases	caught_stealing	\
19370	1.0		3.0		2.0
20810	2.0		36.0		18.0
20811	0.0		0.0		0.0
20815	0.0		1.0		0.0
20816	0.0		0.0		0.0

	zone_rating	name_given	age
19370		Ryan Cole	28
20810		Fernando Jose	24
20811		Fernando Jose	25
20815		James Lewis	23
20816		James Lewis	24

Remaining Fielding Data:

	pos	g	gs	inn_outs	putouts	assists	errors	double_plays	\
15798	P	65		189.0	6	5	0		2
15438	P	20		51.0	0	5	0		0
15799	P	61		171.0	3	4	0		0
23359	P	35	3.0	310.0	9	17	0		2
15437	P	5		54.0	2	7	0		1

	passed_balls	wild_pitches	opponent_stolen_bases	caught_stealing	\
15798					
15438					
15799					

Scores

```
import numpy as np

# Define a function for safe division with error handling
def safe_division(numerator, denominator):
    # Handle case when denominator is 0 or NaN
    return np.where((denominator == 0) | np.isnan(denominator), np.nan, numerator / denominator)

# Calculate the score based on the formula and handle division by zero
c_data['score'] = safe_division(c_data['assists'] + c_data['putouts'] - c_data['opponent_stolen_bases'] - c_data['passed_balls'], c_data['inn_outs'])

# Print first few rows of c_data with the new 'score' column
print("C Data with 'score' column calculated:")
print(c_data.head())
```

C Data with 'score' column calculated:

	pos	g	gs	inn_outs	putouts	assists	errors	double_plays	\
19370	C	9		NaN	55	3	1		1
20810	C	64		1290.0	267	32	4		3
20811	C	2		6.0	2	0	0		0
20815	C	3		21.0	4	0	1		0
20816	C	1		3.0	0	0	0		0

	passed_balls	wild_pitches	opponent_stolen_bases	caught_stealing	\
19370	1.0			3.0	2.0
20810	2.0			36.0	18.0
20811	0.0			0.0	0.0
20815	0.0			1.0	0.0
20816	0.0			0.0	0.0

	zone_rating	name_given	age	score
19370		Ryan Cole	28	NaN
20810		Fernando Jose	24	0.202326
20811		Fernando Jose	25	0.333333
20815		James Lewis	23	0.142857
20816		James Lewis	24	0.000000

Scores

```
# Calculate scores for position players
# Define a function for safe division with error handling
def safe_division(numerator, denominator):
    # Handle case when denominator is 0 or NaN
    return np.where((denominator == 0) | np.isnan(denominator), np.nan, numerator / denominator)

# Calculate the score based on the formula and handle division by zero
fielding_data_sorted['score'] = safe_division((fielding_data_sorted['putouts'] + fielding_data_sorted['assists'] - fielding_data_sorted['errors']), fielding_data_sorted['inn_

# Print first few rows of fielding_data_sorted with the new 'score' column
print("Fielding Data with 'score' column calculated:")
print(fielding_data_sorted.head())
```

Fielding Data with 'score' column calculated:

	pos	g	gs	inn_outs	putouts	assists	errors	double_plays	\
15798	P	65		189.0	6	5	0		2
15438	P	20		51.0	0	5	0		0
15799	P	61		171.0	3	4	0		0
23359	P	35	3.0	310.0	9	17	0		2
15437	P	5		54.0	2	7	0		1

	name_given	age	score
15798	Trevor William	39	0.058201
15438	Yoslan	33	0.098039
15799	Trevor William	40	0.040936
23359	Alan Bernard	26	0.083871
15437	Yoslan	27	0.166667

Seperation

```
summary_stats = {}

# Compute summary statistics for each position
for position, df in {'C': c_data, 'P': p_data, '1B': b1_data, '2B': b2_data, '3B': b3_data, 'SS': ss_data, 'OF': of_data}.items():
    summary_stats[position] = df['score'].describe()

# Create a DataFrame from the summary statistics
summary_df = pd.DataFrame(summary_stats)

# Transpose the DataFrame for better readability
summary_df = summary_df.T

# Display the summary statistics
print("Summary Statistics for Each Position:")
print(summary_df)
```

```
Summary Statistics for Each Position:
count    mean      std      min      25%      50%      75% \
C      2460.0  0.240032  0.070923 -0.333333  0.208969  0.237190  0.266667
P      18095.0  0.059498  0.042496 -0.500000  0.037612  0.056911  0.076923
1B       5032.0  0.342166  0.114090 -0.333333  0.315667  0.340692  0.371475
2B       3377.0  0.176780  0.070016 -0.333333  0.160000  0.179253  0.198693
3B       1306.0  0.087599  0.055265 -0.500000  0.077700  0.092016  0.103133
SS        980.0  0.160863  0.039290 -0.111111  0.158306  0.161926  0.173846
OF       5732.0  0.083827  0.041453 -0.333333  0.069037  0.082841  0.097102
```

```
max
C      1.000000
P      1.000000
1B      3.333333
2B      1.000000
3B      0.555556
SS      0.666667
OF      1.000000
```

```
[6]: # Separate DataFrames for each position
p_data = fielding_data_sorted[fielding_data_sorted['position'] == 'P']

b1_data = fielding_data_sorted[fielding_data_sorted['position'] == '1B']
b2_data = fielding_data_sorted[fielding_data_sorted['position'] == '2B']
b3_data = fielding_data_sorted[fielding_data_sorted['position'] == '3B']
ss_data = fielding_data_sorted[fielding_data_sorted['position'] == 'SS']

# Group LF, RF, CF into OF and remove DH
of_data = fielding_data_sorted[fielding_data_sorted['position'].isin(['LF', 'RF', 'CF', 'OF'])]
of_data = of_data[of_data['position'] != 'DH']

# Print first few rows of each DataFrame
print("P Data:")
print(p_data.head())

print("\n1B Data:")
print(b1_data.head())

print("\n2B Data:")
print(b2_data.head())

print("\n3B Data:")
print(b3_data.head())

print("\nSS Data:")
print(ss_data.head())

print("\nOF Data:")
print(of_data.head())
```

P Data:

	position	g	gs	inn_outs	putouts	assists	errors	double_plays	\
15798	P	65		189.0	6	5	0		2
15438	P	20		51.0	0	5	0		0
15799	P	61		171.0	3	4	0		0
23359	P	35	3.0	310.0	9	17	0		2
15437	P	5		54.0	2	7	0		1

	name_given	age	score
15798	Trevor William	39	0.058201
15438	Yoslan	33	0.098039
15799	Trevor William	40	0.040936
23359	Alan Bernard	26	0.083871
15437	Yoslan	27	0.166667

1B Data:

```
position = position, inn_outs = inn_outs, putouts = putouts, assists = assists, errors = errors, double_plays = double_plays \
```

Position: C
Regression coefficients for C:
Intercept: 0.24466697484431654
Coefficient: -0.00013429095451952962
Mean Squared Error: 0.004515489990152277

Position: P
Regression coefficients for P:
Intercept: 0.04591098082215887
Coefficient: 0.0004703647519776519
Mean Squared Error: 0.0017200530047111002

Position: 1B
Regression coefficients for 1B:
Intercept: 0.32284113433594536
Coefficient: 0.0006424635120882252
Mean Squared Error: 0.012269141572367099

Position: 2B
Regression coefficients for 2B:
Intercept: 0.16391298464012868
Coefficient: 0.0004575888840130101
Mean Squared Error: 0.0034787890116638387

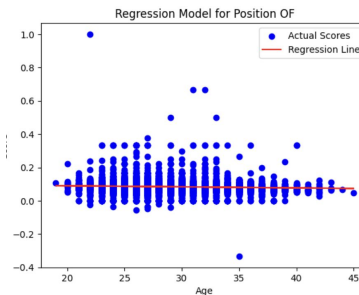
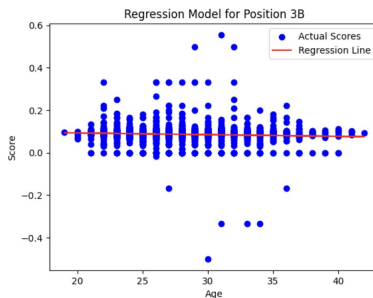
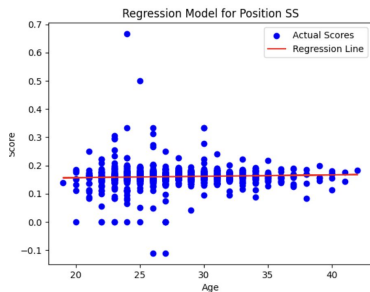
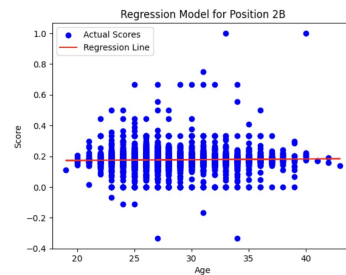
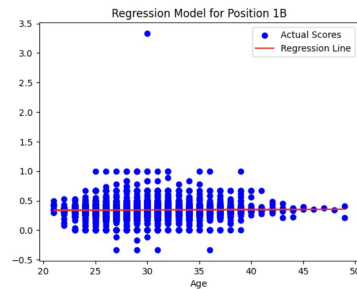
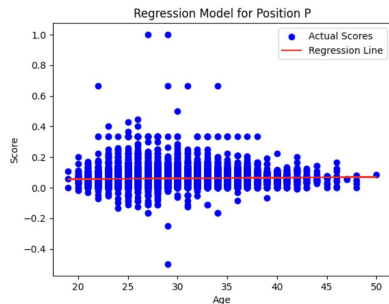
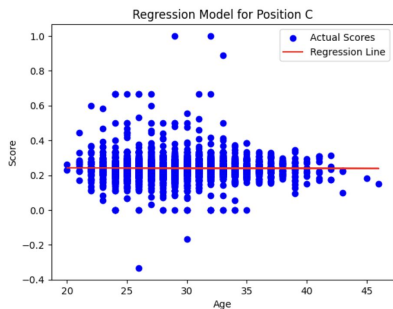
Position: 3B
Regression coefficients for 3B:
Intercept: 0.1091538367962494
Coefficient: -0.0007930734379335808
Mean Squared Error: 0.0033621675409385788

Position: SS
Regression coefficients for SS:
Intercept: 0.14686932662355862
Coefficient: 0.00048718293938758856
Mean Squared Error: 0.0009519096966920087

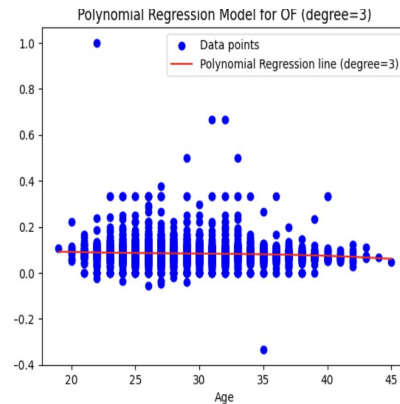
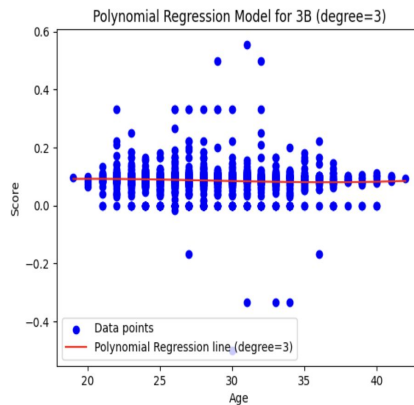
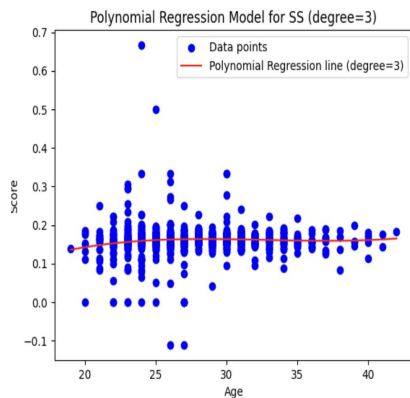
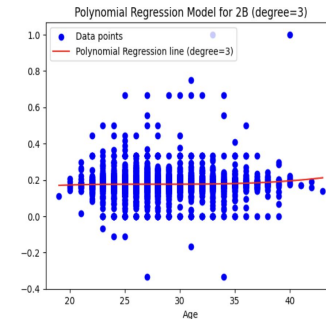
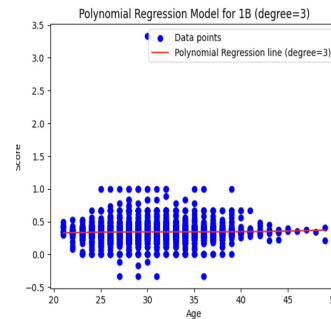
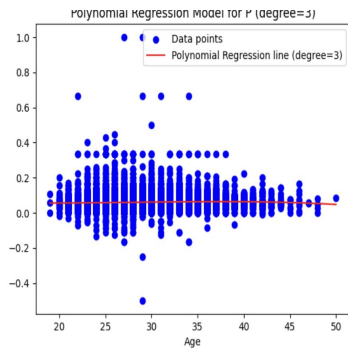
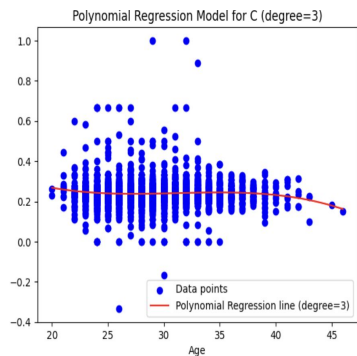
Position: OF
Regression coefficients for OF:
Intercept: 0.10273321551242943
Coefficient: -0.0006452441961604442
Mean Squared Error: 0.0014021912604429484

```
def create_and_evaluate_regression_model(data, position):  
    # Make a copy of the data to avoid modifying the original DataFrame  
    data_copy = data.copy()  
  
    # Drop rows with missing values  
    data_copy.dropna(subset=['score', 'age'], inplace=True)  
  
    # Extract features (X) and target variable (y)  
    X = data_copy[['age']] # Assuming 'age' is the feature  
    y = data_copy['score'] # Assuming 'score' is the target variable  
  
    # Split the data into training and testing sets  
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)  
  
    # Create a Linear Regression model  
    model = LinearRegression()  
  
    # Fit the model to the training data  
    model.fit(X_train, y_train)  
  
    # Evaluate the model on the testing data  
    y_pred = model.predict(X_test)  
    mse = mean_squared_error(y_test, y_pred)  
  
    # Print model coefficients and evaluation metric  
    print(f"Regression coefficients for {position}:")  
    print("Intercept:", model.intercept_)  
    print("Coefficient:", model.coef_[0])  
    print("Mean Squared Error:", mse)  
  
    # Return the trained model  
    return model  
  
# Create and evaluate regression models for each position  
positions = ['C', 'P', '1B', '2B', '3B', 'SS', 'OF']  
models = {}  
  
for position, data in zip(positions, [c_data, p_data, b1_data, b2_data, b3_data, ss_data, of_data]):  
    print(f"Position: {position}")  
    models[position] = create_and_evaluate_regression_model(data, position)  
    print("\n")
```

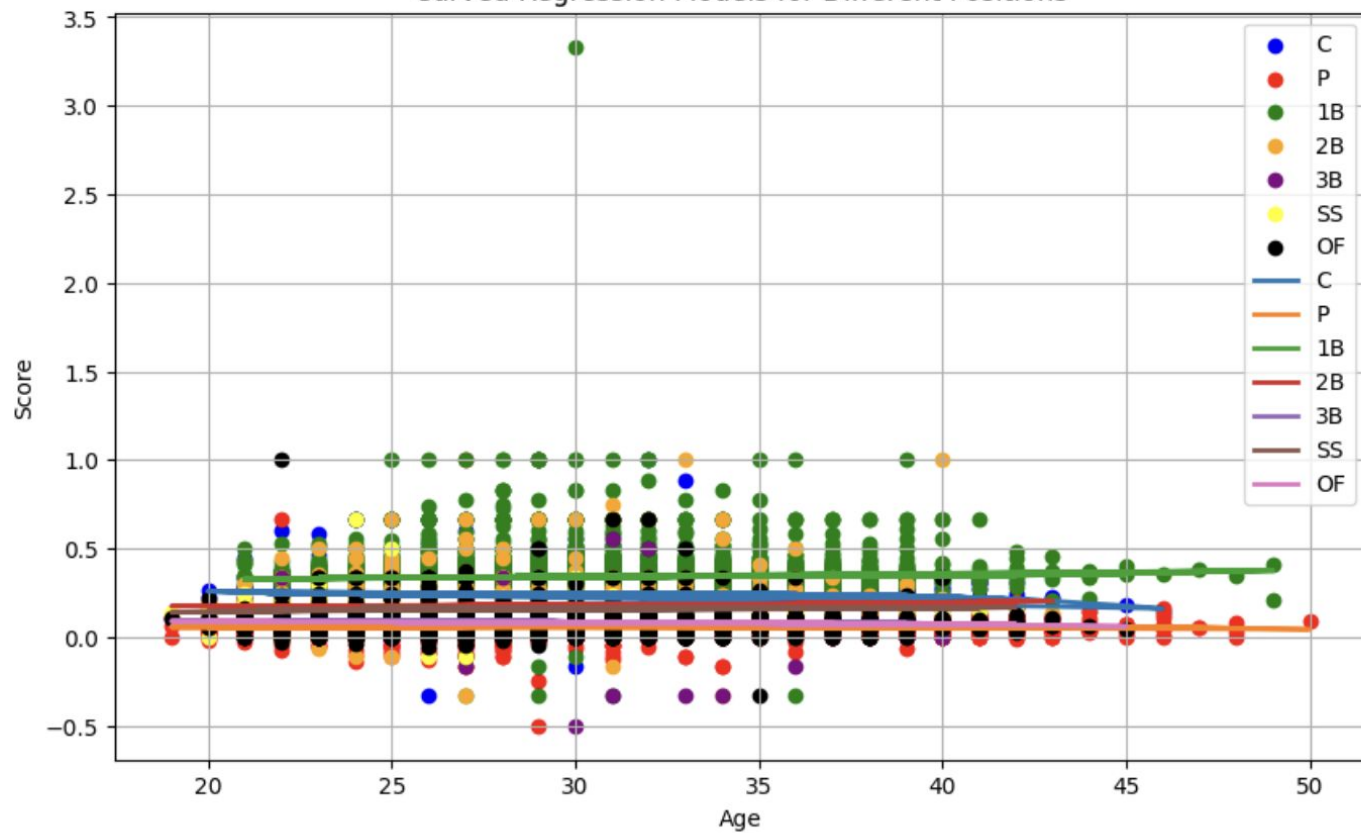
Regression models



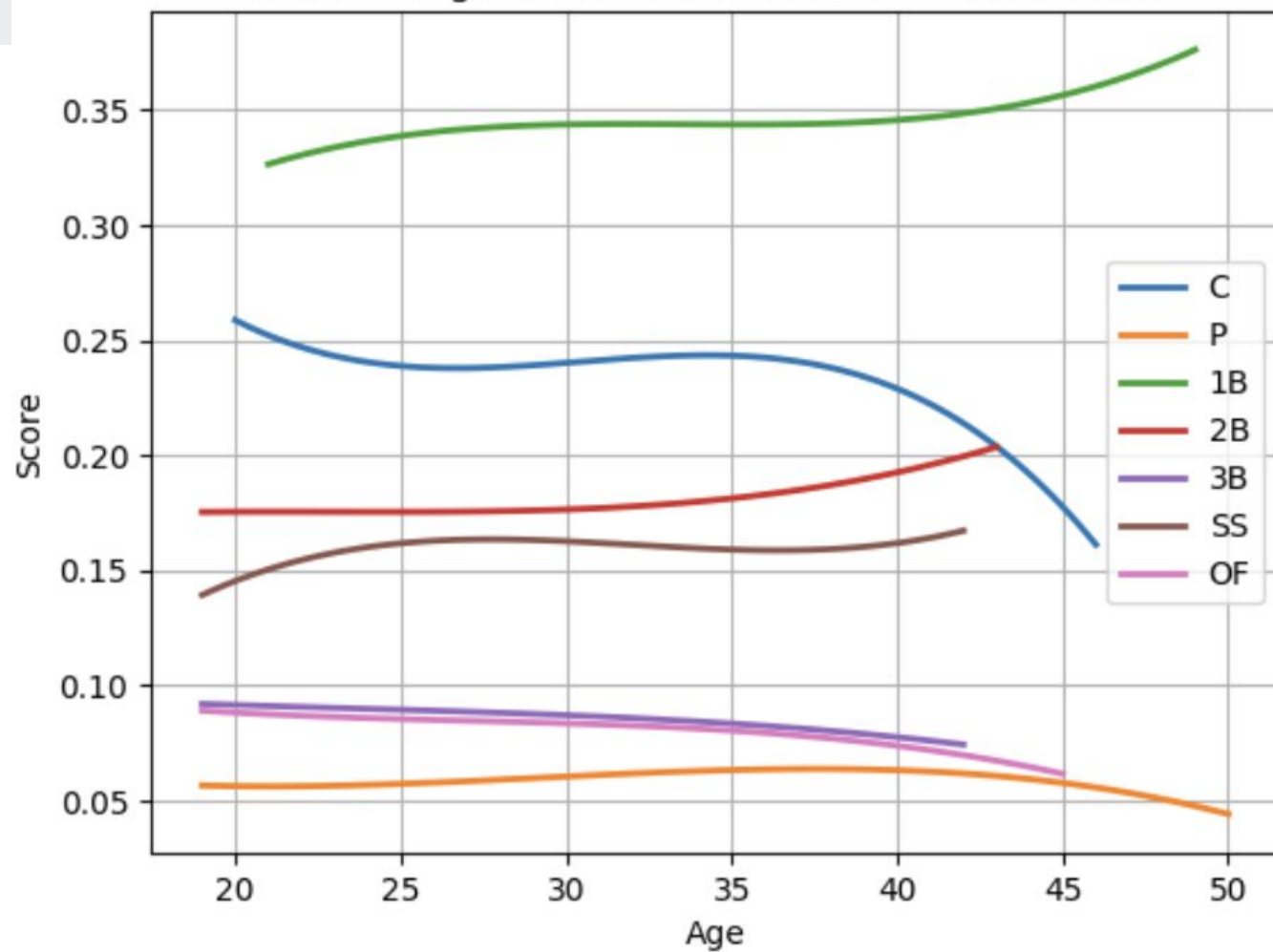
Polynomial Regression



Curved Regression Models for Different Positions



Curved Regression Models for Different Positions



Multiple Regression Model

```
# Split the data into features (X) and target variable (y)
X = combined_data[['age', 'position']]
y = combined_data['score']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('onehot', OneHotEncoder(), ['position'])
    ],
    remainder='passthrough'
)

# Create a pipeline with preprocessing and linear regression model
model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

# Fit the model to the training data
model.fit(X_train, y_train)

# Evaluate the model on the testing data
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

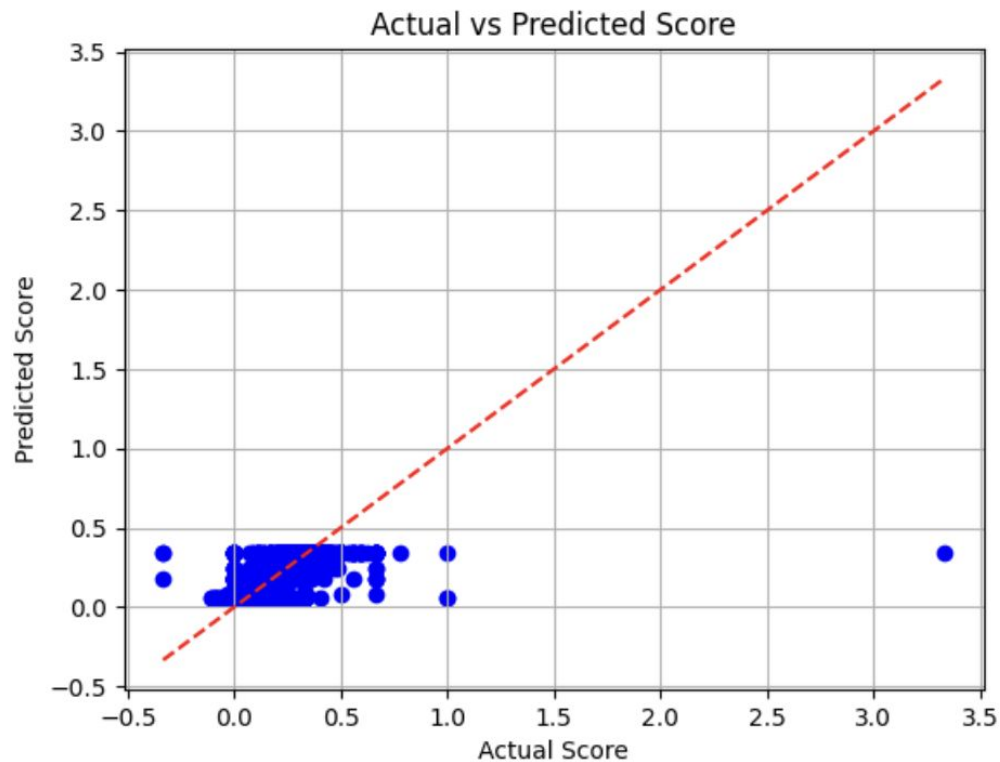
# Print model coefficients and evaluation metric
print("Model Coefficients:")
print(model.named_steps['regressor'].coef_)
print("Intercept:", model.named_steps['regressor'].intercept_)
print("Mean Squared Error:", mse)
```

```
Model Coefficients:
[ 0.17636603  0.01258472 -0.07750667 -0.08024597 -0.10487681 -0.00284039
  0.07651909  0.00023343]
Intercept: 0.15764239107702524
Mean Squared Error: 0.00450618246162656
```

Visualization

```
plt.show()

# Calculate R-squared
from sklearn.metrics import r2_score
r_squared = r2_score(y_test, y_pred)
print("R-squared:", r_squared)
```



R-squared: 0.6882909653134626